

Quiz – Virtualização Containers e Big Data (15 questões)

Selecione todas as alternativas corretas em cada questão. Cada questão pode ter de uma a quatro alternativas corretas.

1. Qual afirmação descreve corretamente a diferença entre máquinas virtuais e containers?

a. Cada VM usa o kernel do host, assim como um container. b. Cada VM possui seu próprio kernel; os containers compartilham o kernel do host. c. O hipervisor isola containers por meio de um kernel novo para cada aplicação. d. Um container precisa iniciar um sistema operacional completo antes de executar a aplicação.

Gabarito comentado

- **a — incorreta:** Uma VM contém um sistema operacional completo, incluindo seu próprio kernel. O compartilhamento do kernel do host caracteriza os containers, por isso aplicar essa característica às VMs inverte a distinção entre as tecnologias.

Referência: 02_VirtualizacaoContainers.pdf — Página 5 do PDF, seção Duas formas de isolar cargas de trabalho. **Evidência no material:** Cada VM possui sistema operacional completo e kernel próprio; os containers compartilham o kernel do host.

- **b — correta:** A VM funciona como um computador independente porque inclui o sistema operacional convidado completo. Já o container separa o ambiente da aplicação mantendo o kernel do host em comum com os demais containers.

Referência: 02_VirtualizacaoContainers.pdf — Página 5 do PDF, seção Duas formas de isolar cargas de trabalho. **Evidência no material:** VMs têm kernel próprio e se comportam como computadores independentes; containers enxergam ambientes separados sobre o mesmo kernel.

- **c — incorreta:** O hipervisor reparte o hardware para executar sistemas operacionais convidados. O isolamento dos containers utiliza mecanismos do kernel Linux, e esses mecanismos não criam outro kernel para a aplicação.

Referência: 02_VirtualizacaoContainers.pdf — Página 6 do PDF, seção O hipervisor: tipo 1 e tipo 2. **Evidência no material:** O hipervisor reparte o hardware e executa vários sistemas operacionais convidados.

Referência: 02_VirtualizacaoContainers.pdf — Página 10 do PDF, seção Os dois pilares: namespaces e cgroups. **Evidência no material:** Namespaces e cgroups repartem o kernel do host e não criam um novo kernel.

- **d — incorreta:** Carregar um sistema operacional inteiro é uma etapa da inicialização de uma VM. O container inicia como processo e utiliza o kernel existente, o que explica o provisionamento mais rápido apresentado na comparação.

Referência: 02_VirtualizacaoContainers.pdf — Página 7 do PDF, seção Características das máquinas virtuais. **Evidência no material:** A VM carrega um sistema operacional inteiro durante o boot.

Referência: 02_VirtualizacaoContainers.pdf — Página 16 do PDF, seção Comparativo geral. **Evidência no material:** O provisionamento de VMs envolve boot do SO; o de containers envolve início de processo.

2. Sobre os tipos de hipervisor e seus ambientes de uso, assinale as afirmações corretas.

a. O hipervisor tipo 1 executa diretamente sobre o hardware. b. O hipervisor tipo 2 dispensa o sistema operacional hospedeiro. c. VirtualBox é um exemplo de hipervisor tipo 2 usado em laboratórios e estudos. d. Instâncias de nuvem como EC2 são exemplos de containers criados por hipervisores tipo 2.

Gabarito comentado

- **a — correta:** O tipo 1 fica diretamente sobre o hardware, sem um sistema operacional intermediário. Essa organização é associada a menor overhead e ao uso em data centers e nuvens públicas.

Referência: 02_VirtualizacaoContainers.pdf — Página 6 do PDF, seção O hipervisor: tipo 1 e tipo 2.

Evidência no material: Tipo 1 é bare metal, executa diretamente no hardware e é usado em data centers e nuvens públicas.

- **b — incorreta:** O tipo 2 é executado como aplicativo dentro de um sistema operacional hospedeiro. Dispensar essa camada intermediária é a característica apresentada para o tipo 1.

Referência: 02_VirtualizacaoContainers.pdf — Página 6 do PDF, seção O hipervisor: tipo 1 e tipo 2.

Evidência no material: Tipo 2 executa como aplicativo sobre Windows, macOS ou Linux.

- **c — correta:** VirtualBox integra os exemplos de hipervisores hospedados. Esse grupo é associado a estudo, laboratório e testes no computador pessoal, pois funciona sobre o sistema operacional hospedeiro.

Referência: 02_VirtualizacaoContainers.pdf — Página 6 do PDF, seção O hipervisor: tipo 1 e tipo 2.

Evidência no material: VirtualBox aparece entre os exemplos de tipo 2, com uso em laboratório, estudo e testes pessoais.

- **d — incorreta:** Uma instância de nuvem é apresentada como servidor virtual alugado: trata-se de uma VM sobre hipervisor tipo 1. A alternativa erra tanto a natureza da instância quanto a classificação do hipervisor.

Referência: 02_VirtualizacaoContainers.pdf — Página 6 do PDF, seção O hipervisor: tipo 1 e tipo 2.

Evidência no material: EC2, Azure VM e Compute Engine exemplificam instâncias que entregam VMs sobre hipervisor tipo 1.

3. Quais características ajudam a explicar o comportamento das máquinas virtuais?

a. A reserva de CPU, memória e disco por VM favorece a previsibilidade dos recursos. b. VMs permitem manter Linux e Windows no mesmo servidor físico. c. O boot de uma VM consiste apenas em iniciar um processo sobre o kernel compartilhado. d. Programas antigos e drivers específicos estão entre os casos de compatibilidade das VMs.

Gabarito comentado

- **a — correta:** A reserva de recursos associa capacidade a cada VM e torna seu uso mais previsível. O material também apresenta a contrapartida dessa organização: maior consumo de recursos.

Referência: 02_VirtualizacaoContainers.pdf — Página 7 do PDF, seção Características das máquinas virtuais. **Evidência no material:** CPU, memória e disco reservados por VM oferecem previsibilidade, com maior consumo.

- **b — correta:** Como cada VM possui seu próprio sistema operacional, sistemas distintos podem coexistir sobre o mesmo servidor físico. Essa flexibilidade também abrange versões diferentes desses sistemas.

Referência: 02_VirtualizacaoContainers.pdf — Página 7 do PDF, seção Características das máquinas virtuais. **Evidência no material:** Linux e Windows, em versões diferentes, podem operar lado a lado no mesmo servidor físico.

- **c — incorreta:** O boot da VM carrega um sistema operacional inteiro, incluindo o kernel convidado. Iniciar um processo sobre kernel compartilhado descreve o funcionamento dos containers, e não a inicialização da VM.

Referência: 02_VirtualizacaoContainers.pdf — Página 7 do PDF, seção Características das máquinas virtuais. **Evidência no material:** A inicialização da VM requer carregar um sistema operacional inteiro.

Referência: 02_VirtualizacaoContainers.pdf — Página 12 do PDF, seção Arquitetura em camadas: VM × Container. **Evidência no material:** A arquitetura de VMs inclui kernels convidados; containers utilizam o kernel compartilhado do host.

- **d — correta:** A compatibilidade ampla permite executar aplicações com poucas ou nenhuma modificação, inclusive programas antigos. A presença de um sistema convidado também atende aos casos de drivers específicos mencionados na aula.

Referência: 02_VirtualizacaoContainers.pdf — Página 7 do PDF, seção Características das máquinas virtuais. **Evidência no material:** A compatibilidade das VMs abrange programas antigos e drivers de hardware específicos.

4. Associe os elementos de um ambiente de containers às suas funções e assinale as associações corretas.

a. Imagem: pacote imutável organizado em camadas de arquivos. b. Runtime: programa que transforma a imagem em um processo em execução. c. Registry: servidor que armazena imagens para operações de pull e push. d. Orquestrador: software que gerencia muitos containers distribuídos entre vários nós.

Gabarito comentado

- **a — correta:** A imagem é o pacote preparado para distribuir a aplicação e suas dependências. Suas camadas formam a base somente de leitura utilizada para iniciar o container, distinguindo o pacote do processo em execução.

Referência: 02_VirtualizacaoContainers.pdf — Página 9 do PDF, seção Conceito de containers.

Evidência no material: A imagem é um pacote pronto, imutável e organizado em camadas.

Referência: 02_VirtualizacaoContainers.pdf — Página 11 do PDF, seção Imagens, camadas e o padrão OCI. **Evidência no material:** As camadas da imagem são somente leitura.

- **b — correta:** O runtime realiza a passagem entre o pacote armazenado e a aplicação efetivamente executada. Containerd e CRI-O são exemplos, e o Docker utiliza containerd internamente.

Referência: 02_VirtualizacaoContainers.pdf — Página 9 do PDF, seção Conceito de containers.

Evidência no material: O runtime transforma a imagem em processo; containerd e CRI-O são exemplos e Docker usa containerd.

- **c — correta:** O registry permite guardar e distribuir imagens. Pull corresponde à obtenção de uma imagem desse servidor, enquanto push corresponde ao envio para ele.

Referência: 02_VirtualizacaoContainers.pdf — Página 9 do PDF, seção Conceito de containers.

Evidência no material: Registry guarda imagens, que são baixadas por pull e enviadas por push.

- **d — correta:** A função do orquestrador envolve coordenar a execução de conjuntos de containers em diferentes nós. Isso amplia a gestão para além de iniciar um único processo a partir de uma imagem.

Referência: 02_VirtualizacaoContainers.pdf — Página 9 do PDF, seção Conceito de containers.

Evidência no material: O orquestrador gerencia muitos containers em muitos nós simultaneamente.

5. Uma equipe reconstrói frequentemente sua imagem e precisa preservar os dados gerados pela aplicação. Quais decisões são adequadas?

a. Guardar dados duráveis na camada gravável, pois ela permanece após docker rm. b. Colocar o código que muda com maior frequência nas últimas camadas da imagem. c. Usar as camadas da imagem como área gravável da aplicação em execução. d. Usar um volume ou armazenamento externo para os dados que precisam durar.

Gabarito comentado

- **a — incorreta:** A camada gravável acompanha o ciclo de vida do container e é apagada quando ele é removido. Dados que precisam sobreviver à remoção devem ser colocados em volume ou armazenamento externo.

Referência: 02_VirtualizacaoContainers.pdf — Página 11 do PDF, seção Imagens, camadas e o padrão OCI. **Evidência no material:** A camada gravável é apagada junto com o container por docker rm.

Referência: 02_VirtualizacaoContainers.pdf — Página 13 do PDF, seção Características dos containers. **Evidência no material:** Dados duráveis devem ir para volumes ou armazenamento externo.

- **b — correta:** O aproveitamento do cache é favorecido pela organização que deixa o conteúdo mais variável ao final. Assim, a reconstrução das camadas alteradas preserva melhor o reaproveitamento das partes anteriores, como dependências.

Referência: 02_VirtualizacaoContainers.pdf — Página 11 do PDF, seção Imagens, camadas e o padrão OCI. **Evidência no material:** A orientação do cache de build é deixar por último o que muda mais, especialmente o código.

- **c — incorreta:** As camadas da imagem são somente leitura. Durante a execução, o container recebe uma camada gravável adicional; ela não transforma as camadas originais em áreas de escrita.

Referência: 02_VirtualizacaoContainers.pdf — Página 11 do PDF, seção Imagens, camadas e o padrão OCI. **Evidência no material:** A imagem possui camadas somente de leitura e o container recebe uma

camada gravável por cima.

- **d — correta:** A persistência deve ser separada da camada efêmera do container. Um volume é uma área gerenciada fora dele, e o armazenamento externo também evita vincular os dados à remoção do container.

Referência: 02_VirtualizacaoContainers.pdf — Página 13 do PDF, seção Características dos containers.

Evidência no material: Volumes são áreas gerenciadas fora do container; volumes ou armazenamento externo recebem dados duráveis.

6. Qual afirmação relaciona corretamente namespaces, cgroups e kernel?

a. Namespaces limitam a quantidade de CPU e memória disponível para o processo. b. Cgroups criam um kernel exclusivo para cada container. c. Namespaces isolam a visão do processo, enquanto cgroups limitam seu consumo de recursos. d. Ao ultrapassar o teto de memória, o processo recebe automaticamente um novo kernel.

Gabarito comentado

- **a — incorreta:** Namespaces definem a visão isolada do processo, como rede e árvore de processos. Os limites de consumo de CPU e memória pertencem aos cgroups, portanto a alternativa troca as funções dos dois mecanismos.

Referência: 02_VirtualizacaoContainers.pdf — Página 10 do PDF, seção Os dois pilares: namespaces e cgroups. **Evidência no material:** Namespaces delimitam o que o processo vê; cgroups limitam CPU, memória e I/O de disco.

- **b — incorreta:** Os cgroups controlam o uso de recursos do processo dentro do kernel existente. Eles não criam kernels convidados, e o container continua compartilhando o kernel do host.

Referência: 02_VirtualizacaoContainers.pdf — Página 10 do PDF, seção Os dois pilares: namespaces e cgroups. **Evidência no material:** Namespaces e cgroups apenas repartem o kernel do host; nenhum cria um novo kernel.

- **c — correta:** A separação de ambientes e a limitação de consumo têm funções complementares. A primeira envolve, por exemplo, interfaces de rede e processos visíveis; a segunda envolve tetos de CPU, memória e vazão de disco.

Referência: 02_VirtualizacaoContainers.pdf — Página 10 do PDF, seção Os dois pilares: namespaces e cgroups. **Evidência no material:** Namespaces isolam processos, rede e outros elementos; cgroups controlam CPU, memória e I/O.

- **d — incorreta:** O teto de memória é aplicado ao processo, e excedê-lo pode provocar seu encerramento pelo kernel, identificado como OOMKilled. A criação de outro kernel não faz parte desse mecanismo.

Referência: 02_VirtualizacaoContainers.pdf — Página 10 do PDF, seção Os dois pilares: namespaces e cgroups. **Evidência no material:** Se o processo ultrapassar o teto de memória, o kernel o encerra com erro OOMKilled.

7. Ao escolher a tecnologia para diferentes cargas de trabalho, quais decisões são coerentes?

a. Priorizar VMs quando a aplicação exige um sistema operacional diferente do host. b. Considerar VMs para sistemas legados difíceis de empacotar em imagens. c. Considerar containers para ETLs que precisam crescer sob demanda e liberar recursos ao terminar. d. Priorizar containers para executar aplicações incompatíveis com o kernel do host.

Gabarito comentado

- **a — correta:** A VM fornece um sistema operacional convidado completo e atende à necessidade de um sistema diferente do hospedeiro. Esse requisito aparece entre os motivos para escolher VMs.

Referência: 02_VirtualizacaoContainers.pdf — Página 17 do PDF, seção Quando escolher VMs.

Evidência no material: Necessidade de outro SO, drivers próprios e ajustes de kernel são critérios favoráveis às VMs.

- **b — correta:** Programas antigos e monolíticos podem exigir ampla reescrita para serem empacotados. A VM é apresentada como escolha para manter esse tipo de aplicação quando a adaptação é difícil.

Referência: 02_VirtualizacaoContainers.pdf — Página 17 do PDF, seção Quando escolher VMs.

Evidência no material: Sistemas antigos e monolíticos difíceis de empacotar são cenários de escolha de VMs.

- **c — correta:** Tarefas em lote e ETLs têm começo e fim e podem apresentar demanda variável. Containers atendem à necessidade de obter recursos rapidamente, escalar e devolvê-los após a conclusão.

Referência: 02_VirtualizacaoContainers.pdf — Página 18 do PDF, seção Quando escolher containers.

Evidência no material: Tarefas em lote, ETL e treinamento crescem sob demanda e devolvem recursos ao terminar.

- **d — incorreta:** O processo do container depende do kernel compartilhado e precisa ser compatível com ele. A necessidade de outro sistema operacional ou kernel orienta a escolha para VMs.

Referência: 02_VirtualizacaoContainers.pdf — Página 16 do PDF, seção Comparativo geral. **Evidência no material:** Containers executam aplicações compatíveis com o kernel do host.

Referência: 02_VirtualizacaoContainers.pdf — Página 31 do PDF, seção Checklist de decisão. **Evidência no material:** A necessidade de outro SO ou kernel específico aponta para VM.

8. Sobre arquiteturas que combinam virtualização e containers, assinale as afirmações corretas.

a. Uma VM pode atuar como nó de um cluster que executa containers. b. Em Kubernetes gerenciado, VMs e containers podem contribuir, respectivamente, com isolamento e agilidade. c. MicroVMs combinam a praticidade dos containers com o isolamento de máquinas virtuais. d. Docker Desktop e WSL2 são apresentados como exemplos de uso de uma VM Linux leve para hospedar containers.

Gabarito comentado

- **a — correta:** O esquema híbrido mostra nós do cluster implementados em VMs e pods executando dentro desses nós. Assim, a virtualização oferece uma camada de isolamento e os containers organizam a execução das aplicações.

Referência: 02_VirtualizacaoContainers.pdf — Página 19 do PDF, seção Na prática: as duas coisas juntas. **Evidência no material:** A arquitetura híbrida mostra VMs como nós do cluster com pods de Spark, Kafka e Airflow.

- **b — correta:** A aula apresenta EKS, AKS e GKE com containers executando dentro de VMs. As camadas têm papéis complementares: a VM isola clientes e o container dá agilidade à execução.

Referência: 02_VirtualizacaoContainers.pdf — Página 19 do PDF, seção Na prática: as duas coisas juntas. **Evidência no material:** Kubernetes gerenciado executa containers em VMs; a VM isola os clientes e o container oferece agilidade.

- **c — correta:** MicroVMs são apresentadas como VMs mínimas que iniciam rapidamente. Essa redução permite combinar a praticidade associada aos containers com a fronteira de isolamento das VMs.

Referência: 02_VirtualizacaoContainers.pdf — Página 19 do PDF, seção Na prática: as duas coisas juntas. **Evidência no material:** MicroVMs iniciam em frações de segundo e unem praticidade de container ao isolamento de VM.

- **d — correta:** O exemplo do computador pessoal mostra que usar containers pode envolver uma camada de virtualização. A VM Linux leve fornece o ambiente no qual esses containers são hospedados.

Referência: 02_VirtualizacaoContainers.pdf — Página 19 do PDF, seção Na prática: as duas coisas juntas. **Evidência no material:** No computador pessoal, Docker Desktop e WSL2 executam uma VM Linux leve para hospedar containers.

9. Qual interpretação é adequada ao comparar desempenho e compatibilidade de VMs e containers?

a. A referência a centenas de containers por host estabelece um limite fixo de capacidade. b. A imagem permite executar a aplicação mesmo quando ela é incompatível com o kernel de destino. c. Containers precisam carregar um kernel convidado, o que explica seu provisionamento em minutos. d. O compartilhamento do kernel reduz overhead, mas exige compatibilidade com o kernel do host.

Gabarito comentado

- **a — incorreta:** Os valores da comparação indicam ordens de grandeza, e não limites garantidos ou fixos. O número viável de instâncias varia conforme o hardware e a carga executada.

Referência: 02_VirtualizacaoContainers.pdf — Página 16 do PDF, seção Comparativo geral. **Evidência no material:** Dezenas de VMs e centenas de containers são ordens de grandeza dependentes do hardware e da carga.

- **b — incorreta:** Empacotar dependências não elimina a relação com o kernel do host. A portabilidade apresentada depende de o destino ter kernel compatível, pois as chamadas de sistema são entregues a ele.

Referência: 02_VirtualizacaoContainers.pdf — Página 12 do PDF, seção Arquitetura em camadas: VM × Container. **Evidência no material:** O container faz chamadas de sistema diretamente ao kernel do host e precisa ser compatível com ele.

Referência: 02_VirtualizacaoContainers.pdf — Página 16 do PDF, seção Comparativo geral. **Evidência no material:** A portabilidade pela imagem é condicionada ao kernel de destino do mesmo tipo.

- **c — incorreta:** Containers não incluem a inicialização de um kernel convidado. A comparação associa minutos ao boot de VMs e segundos ao início de processos em containers.

Referência: 02_VirtualizacaoContainers.pdf — Página 12 do PDF, seção Arquitetura em camadas: VM × Container. **Evidência no material:** Containers não têm kernel convidado.

Referência: 02_VirtualizacaoContainers.pdf — Página 16 do PDF, seção Comparativo geral. **Evidência no material:** VMs são provisionadas em minutos pelo boot do SO; containers em segundos pelo início de processo.

- **d — correta:** Sem carregar um kernel convidado, o container reduz o custo de execução e inicia mais rapidamente. Como usa o kernel do host para suas chamadas de sistema, permanece a exigência de compatibilidade com esse kernel.

Referência: 02_VirtualizacaoContainers.pdf — Página 12 do PDF, seção Arquitetura em camadas: VM × Container. **Evidência no material:** A ausência de kernel convidado reduz overhead e acelera o início, mas as chamadas de sistema dependem do kernel do host.

10. Um serviço foi declarado com três réplicas. Quais afirmações descrevem o papel da orquestração?

- a. O loop de reconciliação procura manter o estado real de acordo com as três réplicas declaradas. b. Scheduling é a função que armazena as imagens em um registry. c. O Service aumenta e reduz a quantidade de nós do cluster. d. A recuperação automática pode reiniciar um container com falha ou recriá-lo em outro nó.

Gabarito comentado

- **a — correta:** No modelo declarativo, a quantidade desejada orienta a atuação contínua do orquestrador. Ele compara a situação real com essa declaração e trabalha para aproximá-las.

Referência: 02_VirtualizacaoContainers.pdf — Página 21 do PDF, seção O que é orquestração?.

Evidência no material: O estado desejado pode declarar três réplicas e o loop de reconciliação procura igualar a realidade à declaração.

- **b — incorreta:** Scheduling é a escolha do nó onde cada container será executado, considerando recursos livres. Guardar imagens para distribuição é a função do registry, não do escalonamento.

Referência: 02_VirtualizacaoContainers.pdf — Página 21 do PDF, seção O que é orquestração?.

Evidência no material: Scheduling decide o nó de execução conforme recursos livres.

Referência: 02_VirtualizacaoContainers.pdf — Página 9 do PDF, seção Conceito de containers.

Evidência no material: Registry é o servidor onde ficam guardadas as imagens.

- **c — incorreta:** O Service distribui requisições entre réplicas saudáveis. A adição ou remoção de nós é atribuída a outro componente, portanto não deve ser confundida com balanceamento de carga.

Referência: 02_VirtualizacaoContainers.pdf — Página 24 do PDF, seção O que a orquestração entrega.

Evidência no material: Service distribui requisições entre réplicas saudáveis; outro componente acrescenta e remove nós do cluster.

- **d — correta:** O cluster verifica periodicamente se os containers respondem. Quando identifica uma falha, pode reiniciar ou recriar a execução, contribuindo para recuperar o serviço.

Referência: 02_VirtualizacaoContainers.pdf — Página 24 do PDF, seção O que a orquestração entrega.

Evidência no material: Containers são verificados periodicamente e os que falham são reiniciados ou recriados em outro nó.

11. Sobre componentes e objetos do Kubernetes, quais associações estão corretas?

a. etcd: armazena o estado declarado do cluster. b. kube-scheduler: escolhe o nó em que cada pod será executado. c. Pod: menor unidade do Kubernetes, com um ou mais containers. d. StatefulSet: organiza cópias com identidade e disco próprios.

Gabarito comentado

- **a — correta:** O etcd integra o control plane e mantém o estado declarado. Esse papel difere da escolha do nó, realizada pelo scheduler, e da comparação entre desejado e real, atribuída ao controller-manager.

Referência: 02_VirtualizacaoContainers.pdf — Página 22 do PDF, seção Kubernetes: arquitetura essencial. **Evidência no material:** No diagrama, etcd guarda o estado declarado; kube-scheduler escolhe nós e controller-manager compara estados.

- **b — correta:** O scheduler participa da distribuição do trabalho selecionando o nó para o pod. Ele não representa o próprio pod nem é o armazenamento do estado declarado.

Referência: 02_VirtualizacaoContainers.pdf — Página 22 do PDF, seção Kubernetes: arquitetura essencial. **Evidência no material:** O kube-scheduler aparece no control plane com a função de escolher o nó de cada pod.

- **c — correta:** Um pod pode reunir containers que executam juntos, sendo a unidade mínima apresentada para o Kubernetes. Por isso, pod e container não são necessariamente uma correspondência de um para um.

Referência: 02_VirtualizacaoContainers.pdf — Página 22 do PDF, seção Kubernetes: arquitetura essencial. **Evidência no material:** Pod é definido como a menor unidade, contendo um ou mais containers.

Referência: 02_VirtualizacaoContainers.pdf — Página 19 do PDF, seção Na prática: as duas coisas juntas. **Evidência no material:** Pod é um ou mais containers que rodam juntos.

- **d — correta:** A identidade e o disco próprios distinguem o StatefulSet do Deployment apresentado na aula. Essa característica atende às cargas que precisam preservar estado, como bancos de dados e Kafka.

Referência: 02_VirtualizacaoContainers.pdf — Página 22 do PDF, seção Kubernetes: arquitetura essencial. **Evidência no material:** StatefulSet possui cópias com identidade e disco próprios; Deployment é apresentado como cópias sem dados próprios.

Referência: 02_VirtualizacaoContainers.pdf — Página 29 do PDF, seção Desafios e cuidados. **Evidência no material:** Bancos de dados e Kafka precisam de identidade fixa e disco próprio, usando StatefulSet e PersistentVolume.

12. Quais descrições das ferramentas de orquestração estão corretas?

a. Docker Swarm é a distribuição corporativa de Kubernetes oferecida pela Red Hat. b. k3s, kind e minikube são opções leves de Kubernetes para laboratório. c. OpenShift acrescenta recursos corporativos, como políticas de segurança e suporte comercial. d. Nomad pode distribuir containers, VMs e programas comuns.

Gabarito comentado

- **a — incorreta:** A distribuição corporativa de Kubernetes apresentada é o Red Hat OpenShift. Docker Swarm é outro orquestrador, integrado ao Docker, que trata vários hosts como um conjunto.

Referência: 02_VirtualizacaoContainers.pdf — Página 23 do PDF, seção Ferramentas de orquestração.

Evidência no material: Docker Swarm vem com Docker; Red Hat OpenShift é a distribuição corporativa de Kubernetes.

- **b — correta:** As três opções aparecem como alternativas de laboratório, enquanto EKS, AKS e GKE são versões gerenciadas por provedores. Essa distinção ajuda a reconhecer os ambientes de uso apresentados.

Referência: 02_VirtualizacaoContainers.pdf — Página 23 do PDF, seção Ferramentas de orquestração.

Evidência no material: k3s, kind e minikube são versões leves para laboratório; EKS, AKS e GKE são opções gerenciadas.

- **c — correta:** OpenShift parte de Kubernetes e agrega recursos voltados ao ambiente corporativo. A aula inclui CI/CD integrado, console para desenvolvedores, políticas de segurança mais rígidas e suporte comercial.

Referência: 02_VirtualizacaoContainers.pdf — Página 23 do PDF, seção Ferramentas de orquestração.

Evidência no material: OpenShift adiciona CI/CD, console para desenvolvedores, políticas de segurança e suporte comercial.

- **d — correta:** O conjunto de cargas apresentado para Nomad vai além dos containers. A ferramenta também distribui máquinas virtuais e programas comuns, conforme a descrição comparativa dos orquestradores.

Referência: 02_VirtualizacaoContainers.pdf — Página 23 do PDF, seção Ferramentas de orquestração.

Evidência no material: Nomad é descrito como orquestrador simples que distribui containers, VMs e programas comuns.

13. Qual afirmação descreve corretamente a execução de um job Spark no Kubernetes?

a. O driver coordena o job, e os executors processam os dados em paralelo. b. spark-submit entrega o job ao registry, que escolhe os nós dos executors. c. A alocação dinâmica mantém a quantidade de executors fixa, independentemente da fila de tarefas. d. Ao terminar o job, os executors continuam reservando os recursos usados no processamento.

Gabarito comentado

- **a — correta:** O driver distribui as tarefas e solicita executors ao cluster. Os executors fazem a leitura e os cálculos paralelos, enquanto o Kubernetes os distribui entre os nós.

Referência: 02_VirtualizacaoContainers.pdf — Página 28 do PDF, seção Como o Spark roda no

Kubernetes. **Evidência no material:** O driver coordena o job e pede executors; os executors leem

dados e calculam em paralelo, distribuídos pelo Kubernetes.

- **b — incorreta:** No fluxo apresentado, spark-submit entrega o job à API do Kubernetes. Registry armazena imagens; a distribuição dos executors pelos nós pertence ao Kubernetes.

Referência: 02_VirtualizacaoContainers.pdf — Página 28 do PDF, seção Como o Spark roda no Kubernetes. **Evidência no material:** spark-submit entrega o job à API Kubernetes e Kubernetes distribui executors pelos nós.

Referência: 02_VirtualizacaoContainers.pdf — Página 9 do PDF, seção Conceito de containers.

Evidência no material: Registry guarda imagens para pull e push.

- **c — incorreta:** A alocação dinâmica permite criar e remover executors conforme a fila de tarefas. Manter sempre uma quantidade fixa contrariaria justamente a adaptação à demanda descrita.

Referência: 02_VirtualizacaoContainers.pdf — Página 28 do PDF, seção Como o Spark roda no Kubernetes. **Evidência no material:** O Spark cria e remove executors conforme o tamanho da fila de tarefas na alocação dinâmica.

- **d — incorreta:** A etapa de limpeza remove os executors e libera seus recursos. Essa liberação é parte do modelo de processamento efêmero e permite devolver a capacidade após a conclusão.

Referência: 02_VirtualizacaoContainers.pdf — Página 28 do PDF, seção Como o Spark roda no Kubernetes. **Evidência no material:** Ao terminar, os executors são removidos e os recursos liberados.

14. Quais cuidados são adequados ao executar aplicações de dados em containers?

a. Planejar identidade fixa e armazenamento persistente para bancos de dados e Kafka. b. Inserir senhas nas camadas da imagem para distribuí-las junto com o código. c. Considerar o tráfego do shuffle do Spark no planejamento da rede. d. Usar imagens confiáveis, verificar vulnerabilidades e evitar privilégios de administrador no container.

Gabarito comentado

- **a — correta:** Essas aplicações precisam conservar estado além da execução de um container. A combinação de StatefulSet e PersistentVolume é apresentada para fornecer identidade e disco próprios, com operadores maduros reduzindo o risco operacional.

Referência: 02_VirtualizacaoContainers.pdf — Página 29 do PDF, seção Desafios e cuidados. **Evidência no material:** Bancos de dados e Kafka precisam de identidade fixa e disco próprio, com StatefulSet e PersistentVolume.

- **b — incorreta:** O conteúdo das camadas acompanha a imagem e pode ser lido por quem a baixa. A orientação é manter senhas fora dela e fornecê-las por Secret no momento da execução.

Referência: 02_VirtualizacaoContainers.pdf — Página 11 do PDF, seção Imagens, camadas e o padrão OCI. **Evidência no material:** O conteúdo das camadas viaja com a imagem e pode ser lido por quem a baixa.

Referência: 02_VirtualizacaoContainers.pdf — Página 29 do PDF, seção Desafios e cuidados. **Evidência no material:** Senhas ficam fora da imagem, em Secret entregue ao container na execução.

- **c — correta:** Shuffle é a troca de dados entre executors e pode produzir muito tráfego. A rede virtual acrescenta atraso, de modo que essa comunicação precisa ser considerada no planejamento do processamento.

Referência: 02_VirtualizacaoContainers.pdf — Página 29 do PDF, seção Desafios e cuidados. **Evidência no material:** Shuffle troca dados entre executors, gera muito tráfego e sofre atraso adicional da rede virtual.

- **d — correta:** As medidas tratam da segurança da cadeia de imagens e das condições de execução. Elas combinam cuidado com a origem, busca por vulnerabilidades e redução dos privilégios do container.

Referência: 02_VirtualizacaoContainers.pdf — Página 29 do PDF, seção Desafios e cuidados. **Evidência no material:** A segurança inclui imagens confiáveis, varredura de vulnerabilidades e containers sem privilégios de administrador.

15. Uma infraestrutura atende várias equipes e pipelines com volumes variáveis de dados. Quais práticas são adequadas?

- a. Reservar capacidade constante para todas as cargas, sem acompanhar as variações de volume. b. Separar equipes por divisões lógicas e cotas de recursos no cluster. c. Combinar processamento efêmero e elástico com armazenamento durável e separado. d. Usar namespaces do Kubernetes como se fossem os mecanismos Linux que isolam a visão de um processo.

Gabarito comentado

- **a — incorreta:** A escalabilidade elástica acompanha o crescimento e a redução do volume de dados. Manter a mesma capacidade sem considerar variações contraria a proposta de evitar recursos ociosos e ajustar o uso à demanda.

Referência: 02_VirtualizacaoContainers.pdf — Página 26 do PDF, seção Benefícios para projetos de Big Data. **Evidência no material:** Recursos crescem e encolhem com o volume de dados para evitar capacidade ociosa.

- **b — correta:** Divisões lógicas organizam o compartilhamento, e cotas limitam quanto cada equipe pode consumir. Juntas, essas medidas ajudam a impedir que uma equipe use todo o cluster e prejudique as outras.

Referência: 02_VirtualizacaoContainers.pdf — Página 26 do PDF, seção Benefícios para projetos de Big Data. **Evidência no material:** Divisões lógicas e cotas evitam que uma equipe consuma o cluster inteiro e prejudique as demais.

- **c — correta:** O processamento pode ser criado, ampliado e encerrado conforme a necessidade. Manter o armazenamento durável separado permite preservar os dados enquanto os recursos de processamento variam ou são liberados.

Referência: 02_VirtualizacaoContainers.pdf — Página 32 do PDF, seção O que levar desta aula.

Evidência no material: A síntese apresenta processamento efêmero e elástico sobre armazenamento durável e separado.

Referência: 02_VirtualizacaoContainers.pdf — Página 28 do PDF, seção Como o Spark roda no Kubernetes. **Evidência no material:** Executors são removidos ao fim do job, liberando recursos.

- **d — incorreta:** Namespaces do Kubernetes são divisões lógicas do cluster para organizar, por exemplo, equipes. Namespaces do Linux isolam a visão dos processos; embora tenham o mesmo nome, o material distingue explicitamente os conceitos.

Referência: 02_VirtualizacaoContainers.pdf — Página 18 do PDF, seção Quando escolher containers.

Evidência no material: Namespaces do Kubernetes são divisões lógicas e não se confundem com os namespaces do Linux.

Referência: 02_VirtualizacaoContainers.pdf — Página 10 do PDF, seção Os dois pilares: namespaces e cgroups. **Evidência no material:** Namespaces do Linux isolam o que o processo consegue ver.